COMP S492F/3920SEF Machine Learning

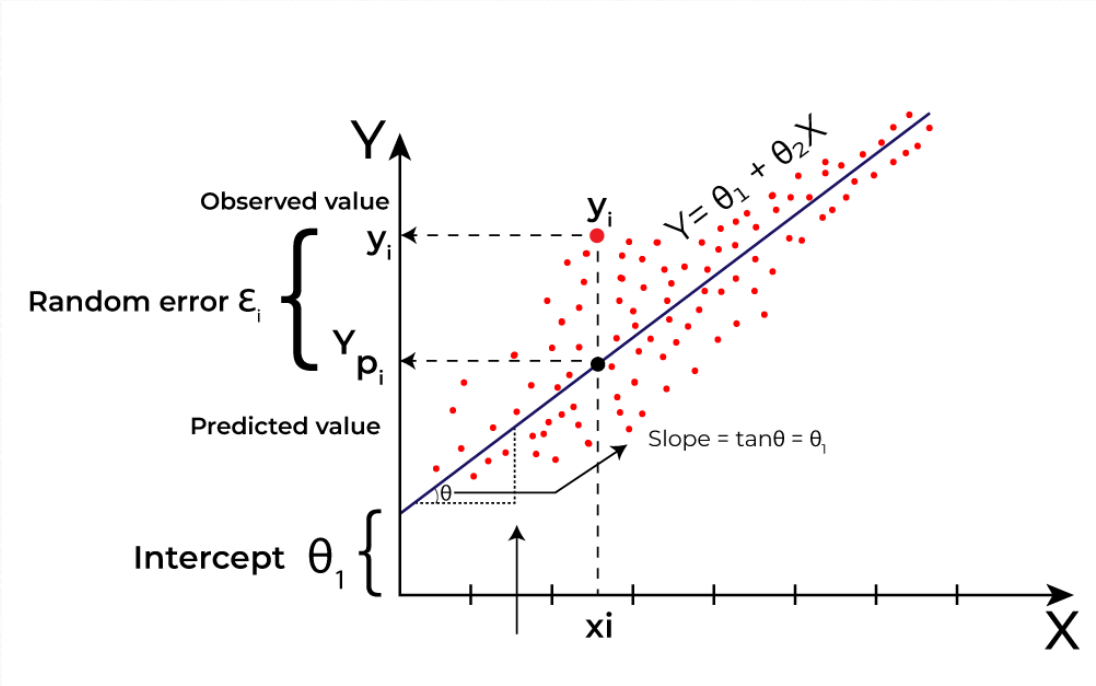
Supervised machine learning 2

Spring 2026
Department of Electronic Engineering and Computer Science (EECS)
School of Science and Technology (S&T)
Hong Kong Metropolitan University

Contents

- Solving linear regression
- Gradient descent
- Regularization
- Parameters

Linear regression



$$f_\theta(x) = \theta_1 + \theta_2 x$$

$$f_\theta(x) = \theta_0 + \theta_1 x$$

$$\boldsymbol{f_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n}$$

$$f_\theta(x) = \theta_0 x_0 + \theta_1 {x_1}^2 + \theta_2 {x_2}^3 + \cdots + \theta_n {x_n}^n$$

$$f_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$f_\theta(x) = \theta^T X$$

where: $\theta = \begin{matrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{matrix}$ $\quad x = \begin{matrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{matrix}$

Linear regression

$$f_\theta(x) = \theta_1 + \theta_2 x$$

$$f_\theta(x) = \theta_0 + \theta_1 x$$

$$\boldsymbol{f_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n}$$

$$f_\theta(x) = \theta_0 x_0 + \theta_1 {x_1}^2 + \theta_2 {x_2}^3 + \cdots + \theta_n {x_n}^n$$

$$f_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$f_\theta(x) = \theta^T x$$

where: $\theta = \begin{matrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{matrix}$ $\quad x = \begin{matrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{matrix}$

So, what we are going to do in linear regression?

Optimization with the function $f_\theta(x) = \theta^T x$

What is our objective here? To minimize the route distance from Yuen Long to Kowloon?

We want to minimize the **error** when we fit the data with the function $f_\theta(x) = \theta^T x$

So how we measure this **error?**

Fitting The Data

Find parameters that reduce the misfit between the model and observed data.

Objective function:

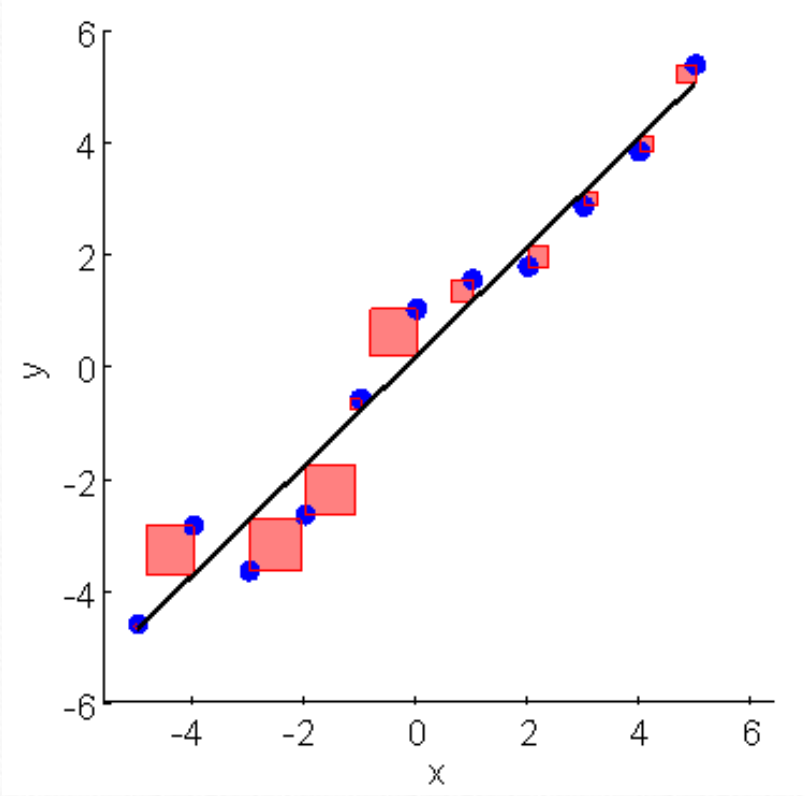
- Error function: Mean Squared Error (MSE) measures the average of the squares of errors with *m* samples in a training set:

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}\left(f_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

- Cost (Loss) function: a measure of the MSE:

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

  - ½ is added for convenience to use the **update rule** derived from gradient descent later.



- By minimizing the cost function, the optimal set of weights can be found.

Linear regression

Feature value in your dataset

Target value in your dataset

$$\min_{\theta} J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{\theta}\left(x^{(i)}\right) - y^{(i)}\right)^2$$

How machine solve this equation?
Does it find out the $Extrema$ Directly?

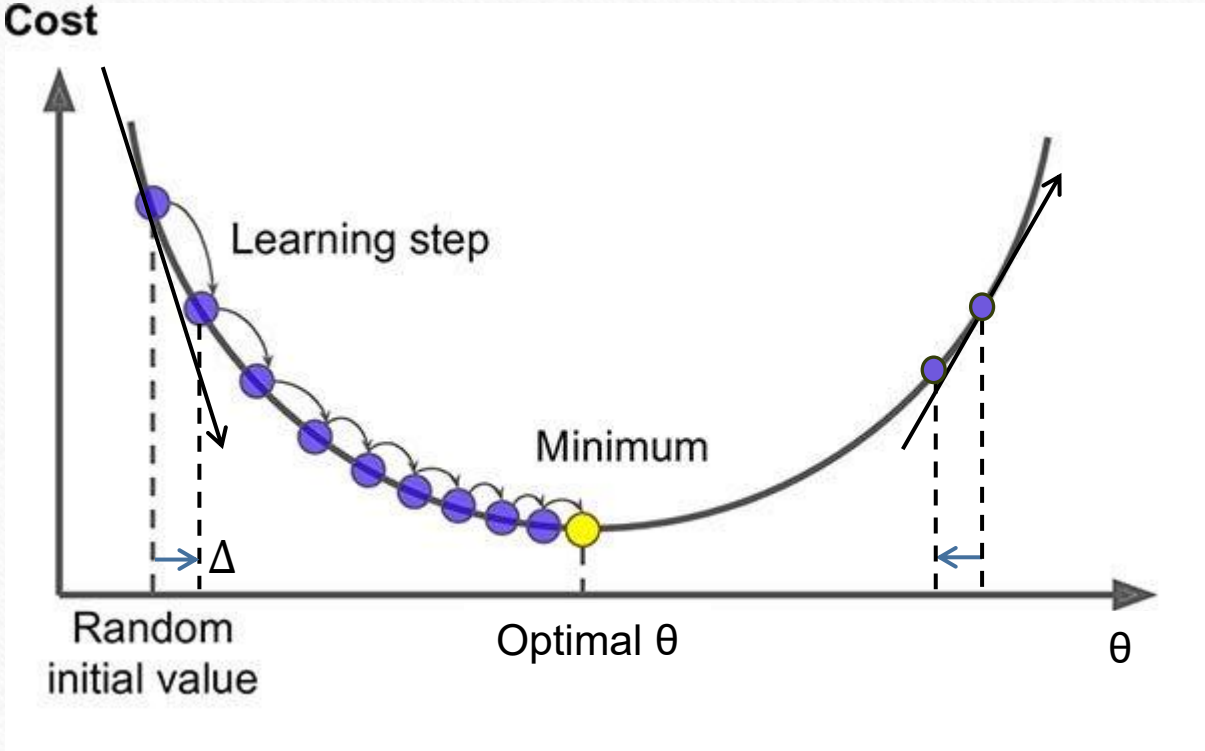
Gradient descent

Gradient descent iteratively updates the weights θ by calculating the partial derivatives of the cost function (pointing to the direction moving towards the minimum) and applying the **update rule** for the weights in each iteration.

The *gradient vector* (i.e. the slope) is denoted by the partial derivatives of the cost function with respect to the weights each of which points in the direction of greatest increase of the cost function with the respective weight.

first derivative is negative
increase θ

first derivative is positive
decrease θ



$\Delta = \alpha \frac{\delta}{\delta \theta_j} J(\theta)$, where α is called learning rate

Gradient Descent and Update Rule

To get the update rule:

- Linear function: $f_\theta\left(x^{(i)}\right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots \theta_n x_n$
- Cost function: $J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(f_\theta\left(x^{(i)}\right) - y^{(i)})^2$
- Partial derivative of cost function: $\frac{\delta}{\delta\theta_j}J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(f_\theta\left(x^{(i)}\right) - y^{(i)})x_j^{(i)}$
- Update rule: $\boxed{\theta_j := \theta_j - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_\theta\left(x^{(i)}\right) - y^{(i)})x_j^{(i)}}$
  - where α is the *learning rate* which is a scaling factor.
  - θ_j (for j = 0, 1, …., n) are updated to θ_j : simultaneously.
- E.g. for n = 2: $\theta_0 := \theta_0 - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(f_\theta\left(x^{(i)}\right) - y^{(i)}\right)$

  $f_\theta(x) = \theta_0 + \theta_1 x_1$ $\quad \theta_1 := \theta_1 - \alpha\frac{1}{m}\sum_{i=1}^{m}(f_\theta\left(x^{(i)}\right) - y^{(i)})x_1^{(i)}$

Test Example

Given the Cost function of a linear regression problem:

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(f_\theta(x^{(i)}) - y^{(i)})^2$$

and the model

$$f_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2$$

Compute the Update Equation

(given that the learning rate = $\propto$)

$$f_\theta(x) = \theta_0 + x_1\theta_1 + x_2^2\theta_2$$

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(f_\theta(x^{(i)}) - y^{(i)})^2$$

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(\theta_0 + \theta_1 {x_1}^{(i)} + \theta_2({x_2}^{(i)})^2 - y^{(i)})^2$$

If we define $u = \theta_0 + \theta_1 {x_1}^{(i)} + \theta_2({x_2}^{(i)})^2 - y^{(i)}$

Then $J(\theta) = \frac{1}{2m}\sum_{i=1}^{m} u^2$

$$\frac{\partial J(\theta_0)}{\partial \theta_0} = \frac{\partial J}{\partial u}\cdot\frac{\partial u}{\partial \theta_0} = \frac{1}{2m}\sum_{i=1}^{m} 2u\cdot 1 = \frac{1}{m}\sum_{i=1}^{m} u\cdot 1$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left(\theta_0 + \theta_1 {x_1}^{(i)} + \theta_2({x_2}^{(i)})^2 - y^{(i)}\right)1$$

$$\frac{\partial J(\theta_1)}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m}(\theta_0 + \theta_1 {x_1}^{(i)} + \theta_2({x_2}^{(i)})^2 - y^{(i)}){x_1}^{(i)}$$

$$\frac{\partial J(\theta_2)}{\partial \theta_2} = \frac{1}{m}\sum_{i=1}^{m}(\theta_0 + \theta_1 {x_1}^{(i)} + \theta_2({x_2}^{(i)})^2 - y^{(i)})({x_2}^{(i)})^2$$

$$\theta_0 = \theta_0 - \alpha\frac{\partial J(\theta_0)}{\partial \theta_0} \qquad \theta_1 = \theta_1 - \alpha\frac{\partial J(\theta_1)}{\partial \theta_1} \qquad \theta_2 = \theta_2 - \alpha\frac{\partial J(\theta_2)}{\partial \theta_2}$$

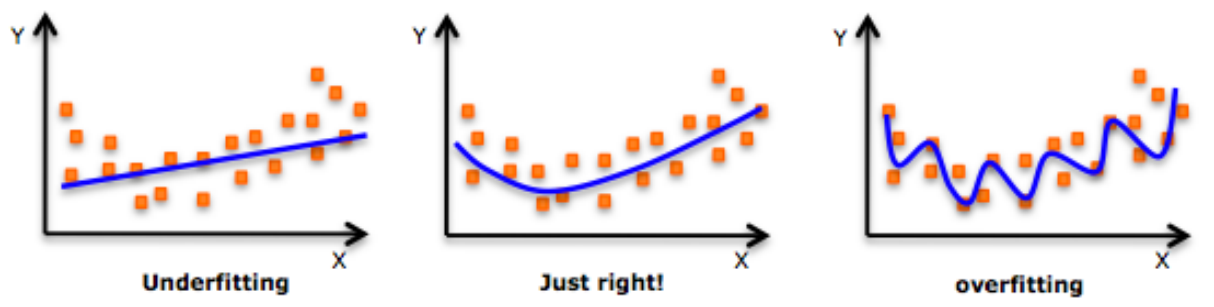
Types of Gradient Descent

- There are 3 types of gradient descent distinguished by the number of training instances that are used for updating the model parameters in each training iteration. One cycle through the entire training dataset is called an epoch.
  - Batch gradient descent (BGD): It uses all of the training instances in each iteration (i.e. an epoch).
  - Stochastic gradient descent (SGD): It uses only a single training instance selected randomly in each iteration (i.e. online training). https://www.youtube.com/watch?v=UmathvAKj80
  - Mini-batch gradient descent (MBGD): It uses a small batch of training instances in each iteration. (nowadays, SGD usually refers to MBGD, unless online training or single instance is emphasized)
- MBGD is often preferred when the number of training instances is large as it converges more quickly than BGD.
- SGD has a noisy gradient signal (due to frequent updates) but it is faster in learning and is applied in deep learning.

Overfitting

- **Overfitting** happens when the model learns signal as well as noise in the training data and wouldn't perform well for new data on which the model wasn't trained before, i.e., it can not generalize the data. The model is more complex and is characterized by large parameters.

Learn features from this cat, then do classification



This is a cat

This is a cat

This is not a cat

Regularization - Ridge Regression

- **To avoid overfitting, regularization is used to penalize high-valued regression coefficients. In other words, it basically adds the penalty as model complexity increases.**
- **In Ridge regression (L2 Regularization), depending on the regularization coefficient λ, all the weights (except the bias) will be penalized/shrink so that the model generalizes the data and won't overfit.**

$$\min_{\theta} J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}\left(f_{\theta}\left(x^{(i)}\right) - y^{(i)}\right)^2 + \boxed{\lambda \sum_{j=1}^{n} \theta_j^2}\right]$$ (Ridge regression/L2 regularization)

Regularization - LASSO regression

- An alternative approach is **LASSO regression (L1 regularization):**

$$\min_{\theta} J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}\left(f_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \boxed{\lambda \sum_{j=1}^{n} |\theta_j|}\right]$$

- LASSO stands for Least Absolute Shrinkage Selector Operator.
- The difference of L1 from L2 is that the regularization term is just the sum of weights instead of the sum of square of weights in L2.
- Those less important weights can become zero (i.e. sparsity), which also makes LASSO useful as a feature selection technique.

Parameters vs. Hyperparameters

Model parameters:

- They are the properties of the training data that are learnt during training, e.g. weight coefficients.
- They differ for each experiment and depend on the type of data and task at hand.

Model hyperparameters:

- They are common for similar models and cannot be learnt during training but are set before training (e.g., learning rate, regularization coefficient, weight initialization scheme, batch size and the number of iterations to be taken).

Batch size: number of training examples utilized in one iteration.

- The learning rate, α, must be set to an appropriate value. Its size determines how fast we will move towards the optimal weights. If α is too large, we might skip the optimal solution. If it is too small, too many iterations is needed to reach convergence.
- In both L1 and L2 regularizations, the model will become overfitting if the regularization coefficient, λ, is too small and underfitting if it is too large.
- The initial random weights should be small (e.g., all set to be zero).

The end